DATA SCIENCE SUBJECT DIGITAL LIBRARY ? STUDY GUIDE

Module 5
Time Series Analysis & Predictive Modeling

An end-to-end study of temporal data, including import and cleaning, visualization, smoothing, correlation, forecasting, predictive model evaluation, and sentiment analysis.

8 teaching periods ? 9 topics ? Learning objectives and full topic notes

Learning Objectives

- Import, inspect, and clean time series datasets from real-world sources.
- Plot time series data and identify trends, cycles, seasonality, and irregular variation.
- Calculate and interpret moving averages for smoothing short-term fluctuations.
- Detect and handle missing values while preserving the temporal order of observations.
- Measure serial correlation and autocorrelation to understand dependence between current and previous observations.
- Explain the predictive modeling lifecycle, including feature preparation, model selection, training, validation, and evaluation.
- Evaluate predictive models using appropriate metrics such as mean absolute error, root mean square error, and coefficient of determination.
- Build practical predictive model solutions for tabular and text-based data.
- Apply sentiment analysis as a text classification problem using predictive modeling techniques.

Contents

1. Time Series Analysis: Importing and Cleaning Data
2. Plotting Time Series and Identifying Temporal Patterns
3. Moving Averages and Smoothing Techniques
4. Handling Missing Values in Time Series
5. Serial Correlation and Autocorrelation
6. Overview of Predictive Modeling
7. Evaluating Predictive Models with MAE, RMSE, and R Squared
8. Building Predictive Model Solutions
9. Sentiment Analysis and Text Classification

Topic Notes

TOPIC 01

Time Series Analysis: Importing and Cleaning Data

A time series is a sequence of data points indexed in chronological order, typically at equally spaced intervals (daily, monthly, yearly). Importing time series data in Python uses pd.read_csv('file.csv', parse_dates=['date_column'], index_col='date_column'), which automatically converts date strings to Pandas DatetimeIndex objects. Cleaning involves: (1) ensuring the datetime index is monotonically increasing with df.sort_index(); (2) converting data types using pd.to_numeric() and pd.to_datetime(); (3) detecting gaps in the time index using df.asfreq('D') which reveals missing dates as NaN rows; and (4) removing duplicate timestamps. Proper datetime indexing unlocks powerful time-based slicing: df['2023':'2024'] selects two full years of data.

TOPIC 02

Plotting Time Series and Identifying Temporal Patterns

Visualizing a time series reveals four classical components: (1) Trend â€” the long-term directional movement (upward or downward) in the data; (2) Seasonality â€” regular, predictable cycles that repeat over a fixed period (e.g., higher retail sales every December); (3) Cyclic patterns â€” irregular fluctuations lasting more than a year, often linked to business or economic cycles; and (4) Irregular/Residual variation â€” random noise remaining after removing trend, seasonality, and cycles. Time series decomposition using statsmodels separates these components into individual plots. Identifying each component is critical because different forecasting methods handle trend and seasonality in different ways.

TOPIC 03

Moving Averages and Smoothing Techniques

A Simple Moving Average (SMA) smooths a time series by replacing each value with the average of a fixed-width window of surrounding values. It removes high-frequency noise, making long-term trends more visible. In Pandas: df['column'].rolling(window=7).mean() computes a 7-period SMA. A larger window produces smoother output but introduces more lag (delayed response to changes). An Exponential Moving Average (EMA) addresses lag by assigning exponentially decreasing weights to older values: df['column'].ewm(span=7).mean(). More advanced smoothing methods include Holt's linear exponential smoothing (which handles trend) and Holt-Winters exponential smoothing (which handles both trend and seasonality).

TOPIC 04

Handling Missing Values in Time Series

Missing values in time series require special care because the temporal order of observations must be preserved â€” randomly deleting rows or filling from a different time period would corrupt the sequence. Common strategies include: (1) Forward fill â€” propagate the last valid observation forward using df.fillna(method='ffill'); appropriate when values change slowly; (2) Backward fill â€” use the next valid value to fill backward; (3) Linear interpolation â€” estimate missing values by drawing a straight line between adjacent known values using df.interpolate(method='linear'); appropriate for smoothly varying data; (4) Seasonal interpolation â€” for strongly seasonal data, use the value from the same period in the previous cycle. The choice of method should reflect the underlying process generating the data.

TOPIC 05

Serial Correlation and Autocorrelation

Serial correlation (autocorrelation) measures the degree to which a time series value at time t is correlated with its own value at an earlier time t-k (lag k). The Autocorrelation Function (ACF) plots the correlation coefficient against lag k, while the Partial Autocorrelation Function (PACF) plots the correlation at lag k after removing the effects of shorter lags. Positive autocorrelation at lag 1 means high values tend to be followed by high values (momentum). Seasonal autocorrelation appears as spikes at multiples of the seasonal period (e.g., lag 12 for monthly data with annual seasonality). The Durbin-Watson statistic (values near 2 = no autocorrelation) is commonly used in regression diagnostics.

TOPIC 06

Overview of Predictive Modeling

Predictive modeling uses historical data to build a mathematical model that forecasts future outcomes or predicts unknown values for new inputs. The predictive modeling lifecycle includes: (1) Problem framing â€” defining the target variable and evaluation metric; (2) Feature engineering â€” creating informative input variables; (3) Train/validation/test split â€” holding out data for unbiased evaluation; (4) Model selection â€” choosing an algorithm appropriate for the data type and problem; (5) Hyperparameter tuning â€” optimizing model configuration using cross-validation; and (6) Deployment â€ serving predictions via an API or dashboard. Avoiding data leakage (accidentally including future information in training) is the most critical practical discipline in predictive modeling.

TOPIC 07

Evaluating Predictive Models with MAE, RMSE, and R Squared

For regression problems (predicting a continuous value), three primary metrics are used. Mean Absolute Error (MAE) is the average magnitude of prediction errors, expressed in the same units as the target â€” it treats all errors equally. Root Mean Squared Error (RMSE) penalizes large errors more heavily than MAE (due to squaring), making it more sensitive to outliers and critical when large errors are especially harmful. R-squared (coefficient of determination) measures the proportion of variance in the target explained by the model â€” R2 = 1 means perfect prediction, R2 = 0 means the model is no better than predicting the mean, and negative R2 means the model is worse than the mean. Always compare metrics against a simple baseline to determine whether the model truly adds value.

TOPIC 08

Building Predictive Model Solutions

Building a practical predictive model solution involves a systematic pipeline: (1) Load and preprocess data (handle missing values, encode categoricals, scale features); (2) Split data using train_test_split from Scikit-Learn; (3) Select and instantiate a model such as LinearRegression, RandomForestRegressor, or GradientBoostingRegressor; (4) Fit the model using model.fit(X_train, y_train); (5) Generate predictions using y_pred = model.predict(X_test); (6) Evaluate using MAE, RMSE, and R-squared; (7) Iterate by trying different models and tuning hyperparameters with GridSearchCV or RandomizedSearchCV. A good model is not just accurate but interpretable, reproducible, and deployable within the constraints of the application.

TOPIC 09

Sentiment Analysis and Text Classification

Sentiment analysis is the task of automatically classifying text (reviews, tweets, comments) as positive, negative, or neutral. It is framed as a supervised text classification problem. The standard pipeline is: (1) Text preprocessing â€” lowercase conversion, punctuation removal, stopword removal, stemming/lemmatization; (2) Feature extraction â€” converting text to numerical vectors using Bag-of-Words (CountVectorizer), TF-IDF (TfidfVectorizer), or dense word embeddings such as Word2Vec, GloVe, or BERT; (3) Classification â€” training a Logistic Regression, Naive Bayes (particularly effective for text), SVM, or fine-tuned transformer model; (4) Evaluation â€” using accuracy, precision, recall, and F1-score, especially important for imbalanced class distributions. Modern approaches use pre-trained language models such as BERT fine-tuned on domain-specific sentiment datasets for state-of-the-art performance.

Module notes: Time series data contains observations ordered over time, so consecutive values may be statistically dependent. Moving averages help reduce random fluctuations, while autocorrelation measures the relationship between current and lagged values. Predictive modeling requires a clear evaluation strategy, and sentiment analysis demonstrates how text can be converted into meaningful labels for classification and prediction.